



ECS 2026

Enhancing SPFx Projects: Harnessing Live Data Integration

Don Kirkham, Microsoft MVP





PREMIUM



PREMIUM PARTNER



TECHNOLOGY PARTNER



DIAMOND



ECS 2026
05-07 MAY | COLOGNE | GERMANY

PLATINUM



GOLD



SILVER



BRONZE



FUTURE MAKER SILVER



FUTURE MAKER BRONZE



MEDIA PARTNER



DON KIRKHAM

MICROSOFT MVP, M365 DEVELOPMENT & MICROSOFT GRAPH



Arlington, Texas, USA



Senior Technology Advisor



<https://donkirkham.com>



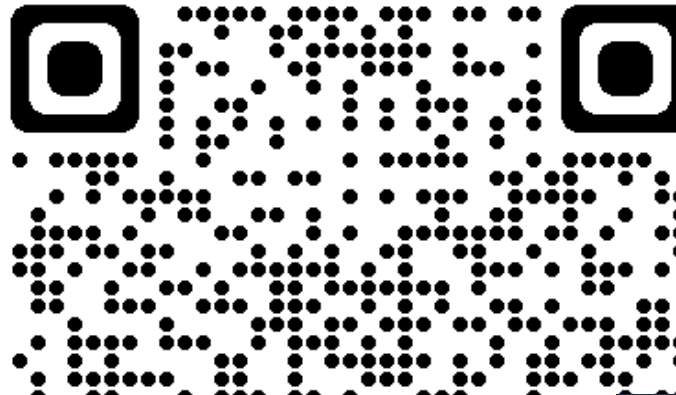
@DonKirkham

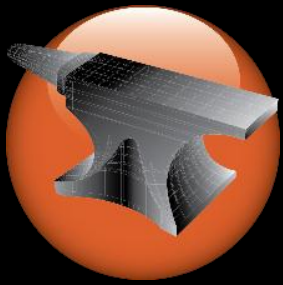


/in/DonKirkham



DonKirkham





SOLUTIONFOUNDRY®

M365 Solutions and Beyond. Built for your needs. By our experts.

Secure Microsoft 365 modernization, automation, and custom development—delivered by our team of senior consultants, developers, and Microsoft MVPs

Core Microsoft 365 services and custom tools

- SPFx Development – SharePoint Framework solutions that extend what M365 can do out of the box.
- SharePoint Modernization – Plan, migrate and transition from Classic SharePoint to the Modern experience.
- Power Platform– Data Visualization with Power BI, workflow automation with Power Automate, and user-friendly Power Apps.
- AI & Copilot Integration – Copilot and other AI tools design and implementation.

Federal & Regulated Program Support

- Decades of experience supporting federal and highly regulated organizations with compliance-driven systems and mission support solutions.
- Primary architects and implementers of enterprise portals, dashboards, and knowledge platforms that standardize processes, improve visibility, and streamline reporting.

General Qualifications

- 3 decades of federal contracting experience
- Microsoft MVP expertise and industry recognition
- Proven ability to embed with customer teams as trusted advisors
- Deep understanding of federal compliance, security, and accountability requirements
- Vast experience with business process, data management and communications analysis, improvement and automation.

Representative Solution Types

- Enterprise portals, dashboards, and reporting for operational visibility
- Compliance workflows, document management, and audit readiness solutions
- Knowledge platforms and self-service support content organized by mission area



Picture time!!!





Why are we here?

DATA IS KING!



Why are we here?

You have several ways to get data into an SPFx web solution

- External APIs via HttpClient (Anonymous and Authenticated)
- SP REST via SPHttpClient
- MS Graph via MSGraphClientV3
- Entra ID Authenticated APIs via AadHttpClient

- PnPjs — wraps the calls above
 - @pnp/queryable
 - @pnp/sp
 - @pnp/graph
 - @pnp/adalclient



Why are we here?

You have several ways to get data into an SPFx web solution

- External APIs via HttpClient (Anonymous ~~and Authenticated~~)
- SP REST via SPHttpClient
- MS Graph via MSGraphClientV3
- ~~Entra ID Authenticated APIs via AadHttpClient~~
- PnPjs — wraps the calls above
 - @pnp/queryable
 - @pnp/sp
 - @pnp/graph
 - ~~@pnp/adalclient~~

Most tutorials pick one and move on. Today we compare them on the same problem.



The roadmap

Intro - Anatomy of a REST call

PASS 1 — SPFx native

- Anonymous calls to a public API
- SP REST against the list
- MS Graph against the same list (and Graph Explorer)

PIVOT

- What are the pain point
- What's missing?

PASS 2 — PnPjs

- Same Anonymous calls to public API
- Same SP operations, fluent
- Same Graph operations, fluent
- The free upgrades!

Wrap — when to still use SPFx-native, decision cheat sheet, Q&A

I n t r o

Anatomy of a REST call

Client-side data interaction: REST APIs

REpresentational State Transfer — the protocol SharePoint, Graph, and most web APIs speak

```
https://mydomain.sharepoint.com/_api/web/lists?$select=Title&$filter=Title eq 'Events'
```

↑ Host ↑ Endpoint ↑ OData query parameters

HTTP Methods

- **GET** — read
- **POST** — create
- **PATCH / MERGE** — update
- **PUT** — replace
- **DELETE** — remove
- **OPTIONS / HEAD** — metadata

Headers

- Authentication (cookies, bearer token)
- Content-Type, Accept, Language

Body (POST / PATCH)

```
{  
  "Title": "New List",  
  "Description": "This is my new list for stuff"  
}
```

■. ■

■ OData Query Operators

```
https://mydomain.sharepoint.com/sites/MySite/_api/web/lists/getbytitle('MyList')/items
```

```
?$select=Id,Title,Description,EventDate  
&$filter=EventDate gt DateTime'2024-01-01T00:00:00'  
&$orderby=EventDate  
&$top=10
```

■ endpoint URL ■ query operators ■ OData operators ■ field names ■ values

P A S S 1

The SPFx Way



SPFx + anonymous APIs

The basic HTTP client in @microsoft/sp-http: HttpClient. No auth. Plain fetch.

```
const response = await this.context.httpClient.get(
  'https://official-joke-api.appspot.com/random_joke',
  HttpClient.configurations.v1
);
const joke: IJokeResponse = await response.json();
return { id: joke.id, setup: joke.setup, punchline: joke.punchline };
```

Use this for any external API that doesn't need your tenant's auth.



SPHttpClient: the mechanics

A fetch wrapper from @microsoft/sp-http, already authenticated to your site

What you get for free

- Auth headers (cookies + bearer)
- ODATA version → 4
- Form digest on writes
- Header defaults

`Accept: application/json;
odata=nometadata`

- Methods for POST operations (merge/update)
- Version Matching commands

What you don't

- Logging
- Caching
- Batching
- Retry
- Type-safe results

SPFx + SP REST: read

```
public async.getItems(list: IListIdentifier): Promise<IEventItem[]> {
  const url = `${this.siteUrl}/_api/web/lists/getbytitle('${list.title}')/items`
    + `?$select=Id,Title,Session,SessionDate,SessionType,EventSite,SessionLink,`
    + `Speaker/Id,Speaker/Title,Speaker/EMail`
    + `&$expand=Speaker`
    + `&$filter=${encodeURIComponent(`SessionDate ge datetime'${startOfTodayIso()}'`)}`
    + `&$orderby=SessionDate asc`;

  const response =
    await this.context.spHttpClient.get(url, SPHttpClient.configurations.v1);
  if (!response.ok) throw new Error(`Failed to get items: ${response.statusText}`);
  const data = await response.json();
  return data.value as IEventItem[];
}
```

It works. It's also a string-concatenated URL, a manual cast, and zero observability.

SPFx + SP REST: update

```
public async updateItem(list, itemId, item): Promise<IEventItem> {
    const url = `${this.siteUrl}/_api/web/lists/getbytitle('${list.title}')/items(${itemId})`;

    const options = {
        headers: { 'IF-MATCH': '*', 'X-HTTP-Method': 'MERGE' },
        body: JSON.stringify(toSpWritePayload(item))
    };

    const response =
        await this.context.spHttpClient.post(url, SPHttpClient.configurations.v1, options);
    if (!response.ok) throw new Error(`Failed to update item ${itemId}: ${response.statusText}`);
    return { ...item, Id: itemId };
}
```

Note the ceremony

- X-HTTP-Method: MERGE — because it's a POST pretending to be a PATCH
- IF-MATCH: '*' — opt out of ETag concurrency
- A separate toSpWritePayload(item) mapper for SharePoint field shapes

SPFx + SP REST: add

Bonus — the cleanest verb in the SP REST set

```
public async createItem(list, item): Promise<IEventItem> {  
    const url = `${this.siteUrl}/_api/web/lists/getbytitle('${list.title}')/items`;  
  
    const options = { body: JSON.stringify(toSpWritePayload(item)) };  
  
    const response =  
        await this.context.spHttpClient.post(url, SPHttpClient.configurations.v1, options);  
    if (!response.ok) throw new Error(`Failed to create item: ${response.statusText}`);  
    return await response.json() as IEventItem;  
}
```

A real POST to the list endpoint. No IF-MATCH. No X-HTTP-Method override. The body is just your fields.

And no, you don't need ListItemEntityTypeFullName.

That was an odata=verbose thing. SPFx defaults to nometadata, so the body is just your fields.

SPFx + SP REST: delete

Bonus — another POST in disguise

```
public async deleteItem(list, itemId): Promise<void> {
    const url = `${this.siteUrl}/_api/web/lists/getbytitle('${list.title}')/items(${itemId})`;

    const options = { headers: { 'IF-MATCH': '*', 'X-HTTP-Method': 'DELETE' } };

    const response =
        await this.context.spHttpClient.post(url, SPHttpClient.configurations.v1, options);
    if (!response.ok) throw new Error(`Failed to delete item ${itemId}: ${response.statusText}`);
}
```

Note the ceremony

- It is a POST. The only thing that turns it into a delete is X-HTTP-Method: DELETE in the headers.
- Same IF-MATCH: '*' opt-out as update.
- Why isn't this spHttpClient.delete()? Because the SP REST endpoint expects POST with the override header. SPFx mirrors what the wire wants.

► Demo 3



When SP REST stops being enough

Reach for MS Graph when...

- You need data outside SharePoint (Outlook, Teams, OneDrive, users, groups)
- You need to span sites or tenants without site-collection-scoped tokens
- You want one API instead of three
 - *Caveat: Graph for SharePoint list items is awkward. You'll see why in a minute.*



The Graph Explorer detour

Same SPFx MSGraphClientV3, no list at all — just path → JSON

```
public async runQuery(path: string): Promise<unknown> {  
    return await this.graphClient.api(path).version('v1.0').get();  
}
```

Use this in real life when prototyping or exploring a Graph endpoint before you commit to a service implementation. Cheaper than Postman because the auth is already there.

SPFx + Graph: read

```
const qs = [  
  `expand=fields(select=${GRAPH_FIELD_SELECT})`,           // <-- no $ on expand  
  `$filter=fields/SessionDate ge '${startOfTodayIsoNoMs()}'`, // <-- single-quoted ISO  
  `$orderby=fields/SessionDate asc`  
].join('&');  
  
const response = await this.graphClient  
  .api(`/sites/${this.siteId}/lists/${list.id}/items`)  
  .version('v1.0')  
  .header('Prefer', 'HonorNonIndexedQueriesWarningMayFailRandomly')  
  .query(qs)  
  .get();
```

Three Graph-specific gotchas already on this slide

1. `expand=fields(select=...)` — no \$. With `$expand` you get an OData parser error.
2. Datetime literal must be a single-quoted ISO 8601 string. `datetime'...'` returns 400.
3. `$filter` / `$orderby` against `fields/<column>` requires the column to be indexed.

SPFx + Graph: add

Bonus — a real POST, plus the fields envelope

```
public async createItem(list, item): Promise<IEventItem> {
  const response = await this.graphClient
    .api(`/sites/${this.siteId}/lists/${list.id}/items`)
    .version('v1.0')
    .post({ fields: toGraphWriteFields(item) });
  return fromGraphItem(response as IGraphListItem);
}
```

A real `.post()` — no method-override gymnastics, no IF-MATCH.

But the `{ fields: ... }` envelope is mandatory

- Graph won't accept a flat field bag — it wants the wrapper.
- `toGraphWriteFields` does the same translations as the SP REST mapper (Speaker → SpeakerLookupId, URL → { Url, Description }).
- Mapping ceremony is a SharePoint shape problem, not an HTTP-client problem.

SPFx + Graph: delete

Bonus — the cleanest delete in the deck

```
public async deleteItem(list, itemId): Promise<void> {  
    await this.graphClient  
        .api(`/sites/${this.siteId}/lists/${list.id}/items/${itemId}`)  
        .version('v1.0')  
        .delete();  
}
```

A real .delete(). No POST. No X-HTTP-Method. No IF-MATCH.

*The SP REST workaround is REST-the-protocol's fault,
not Graph's.*



Pivot: notice anything?

Look at what we wrote across three services

	Logging	Caching	Batching	Retry	Types
HttpClient	×	×	×	×	cast
SPHttpClient	×	×	×	×	cast
MSGraphClientV3	×	×	×	×	cast

Every empty cell is a problem you'd solve yourself.
PnPjs already solved them.

P A S S 2

The PnPjs Way



What PnPjs is

A fluent, composable HTTP pipeline for SharePoint, Graph, and arbitrary endpoints

Three packages today

- @pnp/sp — SharePoint REST, fluent
- @pnp/graph — MS Graph, fluent
- @pnp/queryable — the underlying pipeline (also works against any URL)

Init pattern (once per service)

```
const sp      = spfi(siteUrl).using(spSPFx(this.context));  
const graph = graphfi().using(graphSPFx(this.context));
```

Same SPFx context, same auth. PnPjs is not a separate auth story — it rides on top of the SPFx clients you already have.

The URL reveal

SPFx-native

```
spHttpClient.get(  
  `${siteUrl}/_api/web/lists/  
  + `getbytitle('Speaking Events')/items`  
  + `?$select=Id,Title,SessionDate`  
  + `&$orderby=SessionDate desc&$top=50`,  
  SPHttpClient.configurations.v1  
);
```

PnPjs

```
sp.web.lists.getByTitle('Speaking  
Events').items  
  .select('Id', 'Title', 'SessionDate')  
  .orderBy('SessionDate', false)  
  .top(50)();
```

Both produce the same network request.

PnPjs is not magic.

It's the URL you'd write anyway, just typed for you.



PnPjs + anonymous

```
const q = new Queryable('https://official-joke-api.appspot.com/random_joke');  
q.using(BrowserFetch(), RejectOnError(), ResolveOnData(), JSONParse());  
const joke: IJokeResponse = await q();
```

The same pipeline that talks to SharePoint can talk to anything. You just compose the behaviors you need.

Real-world note: strip X-PnPjs-RequestId on external calls to avoid CORS preflight — see the source.

PnPjs + SP REST: read

```
public async.getItems(list): Promise<IEventItem[]> {  
    return await this.sp.web.lists  
        .getByTitle(list.title)  
        .items  
        .select('Id', 'Title', 'Session', 'SessionDate', 'SessionType',  
            'EventSite', 'SessionLink',  
            'Speaker/Id', 'Speaker/Title', 'Speaker/EMail')  
        .expand('Speaker')  
        .filter(`SessionDate ge datetime'${startOfTodayIso()}'`)  
        .orderBy('SessionDate', true)() as IEventItem[];  
}
```

Same fields. Same filter. Same order.

No URL string. No response.ok check. No response.json().

PnPjs + Graph: read

```
const itemsQuery = this.graph.sites
  .getById(this.siteId)
  .lists
  .getById(list.id)
  .items
  .using(InjectHeaders({ Prefer: 'HonorNonIndexedQueriesWarningMayFailRandomly' }));

itemsQuery.query.set('expand', `fields(select=${GRAPH_FIELD_SELECT})`);
itemsQuery.query.set('$filter', `fields/SessionDate ge '${startOfTodayIsoNoMs()}'`);
itemsQuery.query.set('$orderby', 'fields/SessionDate asc');

const items = await itemsQuery() as IGraphListItem[];
```

Honesty check

The Graph SP-list awkwardness is a Graph problem, not a PnPjs problem. Both clients have to deal with it. PnPjs at least gives you InjectHeaders and a query builder.



PnPjs + SP REST: update

```
public async updateItem(list, itemId, item): Promise<IEventItem> {  
    await this.sp.web.lists  
        .getByTitle(list.title)  
        .items  
        .getById(itemId)  
        .update(toSpWritePayload(item));  
    return { ...item, Id: itemId };  
}
```

Compare to SPFx

- No IF-MATCH header.
- No X-HTTP-Method: MERGE.
- No URL.

Same toSpWritePayload mapper — that's a SharePoint shape problem, not an HTTP-client problem.

PnPjs: add

Bonus — same .add() across SP and Graph

PnPjs SP

```
const result = await this.sp.web.lists
  .getByTitle(list.title)
  .items
  .add(toSpWritePayload(item));
```

PnPjs Graph

```
await this.graph.sites
  .getById(this.siteId)
  .lists.getById(list.id)
  .items
  .add({ fields: toGraphWriteFields(item)
});
```

Same .add() verb across both APIs.

The only difference is the path-builder chain — and that's the API surface, not the HTTP layer.

Note: the Graph version still needs the { fields: ... } envelope — PnPjs hides transport ceremony, not API shape.

PnPjs: delete

Bonus — same .delete() across SP and Graph

PnPjs SP

```
await this.sp.web.lists
  .getByTitle(list.title)
  .items
  .getItemById(itemId)
  .delete();
```

PnPjs Graph

```
await this.graph/sites
  .getItemById(this.siteId)
  .lists.getItemById(list.id)
  .items
  .getItemById(itemId.toString())
  .delete();
```

No URLs. No IF-MATCH. No X-HTTP-Method.

PnPjs normalizes the verb across SP REST (which fakes DELETE via POST) and Graph (which uses real DELETE). Same call site, different wire format.



Free Upgrades: LOGGING

You are hopefully doing it anyway. Let PnPjs supercharge it.

```
Logger.attachConsoleListener();  
Logger.setLevel(this.properties.enhancedLogging ? LogLevel.Verbose : LogLevel.Warning);
```

```
[DataDemo] onInit: starting (enhancedLogging=on)  
[DataDemo] loadItems: requesting items from list Speaking Events  
[DataDemo] loadItems: received 12 item(s) ▶ (12) [{...}, {...}, ...]
```

- A thin wrapper over @pnp/logging (info / debug / warn / error) emits via console
- Object payloads render as collapsible trees.
- Listeners are pluggable — ConsoleListener today, App Insights tomorrow. Same call sites.

Free Upgrades: CACHING

Two scopes. Pick one.

```
// Global: every read on this SPFI is cached.
const sp = spfi(siteUrl)
  .using(spSPFx(this.context))
  .using(Caching({ store: 'session' })); // 'session' | 'local'

// Per-call: opt one query out of caching when global is on.
.items.using(CacheNever())()

// Per-call: opt one query in with a custom expiry when global is off.
.items.using(Caching({ store: 'session', expireFunc: () => addMinutes(new Date(), 5) })))()
```

Defaults: 60-second expiry, keyed by URL. 'session' survives a hard refresh; 'local' survives a tab close. Default is in-memory and dies with the page.

Use it for: reference data · repeated reads · anything you'd hate to re-fetch on tab switch

Avoid for: frequently-changing transactional data · anything where stale = wrong (use CacheNever() there)

Free Upgrades: BATCHING

Reads, in our demo. Writes work the same way.

```
const [batchedSp, execute] = this.sp.batched({ maxRequests: 100 });
const pagePromises: Promise<IEventItem[]>[] = [];
for (let i = 0; i < pageCount; i++) {
  pagePromises.push(
    batchedSp.web.lists.getByTitle(list.title).items
      .select('Id', 'Title', 'SessionDate', /* ... */)
      .orderBy('SessionDate', true)
      .top(pageSize).skip(i * pageSize()) as Promise<IEventItem[]>
  );
}
await execute();
const items = (await Promise.all(pagePromises)).flat();
```

100 paginated reads → one HTTP request to `/_api/$batch`. The same wiring works for add/update/delete.

Why it matters

- Latency: one round trip beats N. Especially over slow links.
- Throttling: SharePoint counts requests, not inner operations. Your throttle escape hatch.
- Atomicity-ish: not a real transaction, but closer than N independent calls.

► Demo 7

W R A P

When and Why



When to still reach for SPFx-native

PnPjs is the default. But it's not free.

- Bundle size matters. PnPjs adds ~50–80 KB depending on imports. For a single-purpose tile that does one Graph call, that's a lot.
- One-off calls with no list interaction (a single /me ping, a webhook trigger).
- Debugging a network issue — fewer abstractions between you and the wire.
- Your team doesn't know PnPjs yet and the call is trivial.

Default: PnPjs.

Exceptions: above.

Decision cheat sheet

Scenario	Use
List CRUD on the current site	PnPjs SP
List CRUD across sites (one tenant)	PnPjs Graph
Cross-tenant or app-only	PnPjs Graph (with custom auth)
One-off anonymous fetch	SPFx HttpClient
Many anonymous fetches, want consistency	PnPjs Queryable
Prototyping a Graph endpoint	SPFx MSGraphClientV3 (or Graph Explorer)
Single-purpose perf-critical tile	SPFx native
You need logging/caching/batching	PnPjs (every time)



Three takeaways

1. PnPjs makes the same HTTP request, just typed for you.
2. Logging, caching, and batching are one toggle each.
3. Reach for SPFx-native deliberately, not by default.

Resources

- PnPjs docs — <https://pnp.github.io/pnpjs/>
- SPFx HTTP clients — <https://learn.microsoft.com/sharepoint/dev/spfx/web-parts/basics/connect-to-sharepoint>
- Microsoft Graph — <https://learn.microsoft.com/graph/>
- This demo project — <https://github.com/DonKirkham/DataDemo>

THANK YOU,
YOU ARE AWESOME ❤️

PLEASE RATE THIS SESSION
IN THE MOBILE APP.



<https://donkirkham.com>



@DonKirkham



/in/DonKirkham



DonKirkham

